

Week 4 Code Review

Lessons Learned · Agents, Authentication, and Persistence

This handout collects the recurring themes from this week's group code reviews. It builds on the Week 1–3 guidelines and focuses only on what is new: agentic systems, real authentication, real database persistence, ORM and migrations, async HTTP clients, agent tracing, and the API contracts that hold all of it together. Topics already covered earlier in the bootcamp (logging, project structure, defending your code, RAG basics, Docker volumes, ML hyperparameter tuning) are not repeated here — go back to the previous guides for those.

1. Authentication & Authorization

Most of the new questions this week were about identity: who is the caller, how do we know, and what are they allowed to do. Tokens, headers, and status codes all have specific meanings — vague answers here are a tell that the auth layer was copied without being understood.

What a token is actually for

A token is a credential the client carries on every request after logging in. The server signed it, so the server can trust it without re-checking the password each time. Being able to say that out loud — without hand-waving — is the bar.

JWTs in particular carry a small payload (usually the user id and an expiry) signed with a secret. The signature is what makes them tamper-proof, not the contents — JWT payloads are base64, not encrypted. Do not put anything sensitive inside.

***Likely questions:** What is the use of the token for the user? · What is inside a JWT? · Why is the secret never committed?*

How the token is sent

JWTs travel in the **Authorization** HTTP header with the **Bearer** scheme: `Authorization: Bearer <token>`. Not in the body, not in a cookie unless you have a deliberate reason. Reviewers will ask you to show the exact header in your client and the exact dependency that parses it on the server.

***Likely questions:** How is the JWT token sent? · Where in your FastAPI code is the Authorization header read?*

Token expiry and refresh tokens

Access tokens are short-lived on purpose. When one expires, the user should not have to log in again — that is what a **refresh token** is for. The refresh token lives longer, is stored more carefully, and is exchanged for a new access token when the old one dies. If your project does not implement refresh, you should at least be able to describe the mechanism.

***Likely questions:** What happens when the token expires? · What is a refresh token and where is it stored?*

401 vs 403 — they are not the same

401 Unauthenticated means we do not know who you are — no token, expired token, bad signature. **403 Forbidden** means we know who you are, and you are not allowed to do this. Returning 403 for a missing token (or 401 for a permission failure) is a small bug that loudly tells reviewers you copied the auth code.

***Likely questions:** What is the difference between 401 and 403? · Which status code does your app return when no token is sent?*

Did you actually implement auth?

If your project has user-scoped data (agent runs, saved chats, anything) but no authentication layer, that is a gap, not a feature. Be ready to point at the route dependency that protects each endpoint and the function that issues tokens at login.

***Likely questions:** Did you implement auth in your project? · Show me a protected endpoint and the dependency that protects it.*

2. Database Persistence — the path from request to row

A recurring failure mode this week: students claim data is being persisted, but cannot show the line of code where the write happens, and cannot prove the data survives a container restart. Persistence is not a vibe — it is a sequence of specific calls.

Trace a registration end-to-end

When a user registers, you should be able to walk through, in order: the route that receives the request, the Pydantic model that validates it, the service or CRUD function that builds the ORM object, **the exact line where `session.add()` and `session.commit()` happen**, and the response model that hides sensitive fields on the way out. If any of those steps are missing or fuzzy, the persistence layer is not yours yet.

If the data only exists in a Python list or dict, it is in memory, not in the database. Restart the container and the data will be gone. Test this before the review.

***Likely questions:** How is registration happening — what changes in the code, the database, and the response? · In which function is the data saved in PostgreSQL when a user registers? · Show me the line where the database write actually happens. · Are credentials being saved in the database or in memory? Prove it.*

Verify with pgAdmin (or equivalent)

pgAdmin is a browser-based admin UI for PostgreSQL. Connect it to your database container, open your table, and see the rows. "It worked, the API returned 200" is not the same as "the row is in the table." During a review, open pgAdmin and show the row.

Other clients work too — DBeaver, TablePlus, the `psql` CLI. Pick one and have it ready before the session.

***Likely questions:** Open your database in a client and show me the user you just registered.*

SQLAlchemy + Alembic: the workflow

We do not write raw SQL in this project. We write Python classes that inherit from `Base` (SQLAlchemy ORM), and Alembic generates and applies the migrations that translate those classes into PostgreSQL tables. The split matters:

- **SQLAlchemy** is the ORM — it lets you query and write with Python objects instead of SQL strings.
- **Alembic** is the migration tool — it tracks schema versions, with `upgrade()` applying a change and `downgrade()` reverting it.

Schemas change over the life of an application — new columns, new tables, dropped fields. Migrations are how you change the database safely without dropping it and starting over. "I just deleted the volume" is not a migration strategy.

***Likely questions:** What is Alembic and what does it do that SQLAlchemy does not? · What are migrations and why do you need them? · What does the downgrade method do?*

Tables, classes, and `__tablename__`

Every ORM class maps to one database table. The `__tablename__` attribute is how you tell SQLAlchemy what to call that table in PostgreSQL. If you have a `Chunk` class with `__tablename__ = "chunks"`, that is the line that connects your Python model to the actual table.

***Likely questions:** Why does your `Chunk` class have `__tablename__`? · Where in the database does this class live?*

Relationships — model the real world

If a user can have many agent runs, that is a one-to-many relationship and it should be expressed in the models — a `user_id` foreign key on `AgentRun`, and a `relationship()` on `User`. This is what lets you fetch a user's history with one query instead of stitching it together by hand.

***Likely questions:** What is the relationship between `User` and `AgentRun` in your schema? · Show me the foreign key.*

3. Lifespan, Sessions, and Dependency Injection

FastAPI gives you specific tools for managing resources that live longer than a single request — database engines, HTTP clients, model loads. If you treat these as global module-level variables, things will work right up until they do not.

What `lifespan.py` is for

The `lifespan` function is FastAPI's startup-and-shutdown hook. It runs once when the app boots (set up the database engine, warm up the embedding model, open a shared HTTP client) and once when it shuts down (close everything cleanly). The `@asynccontextmanager` decorator is what makes the before-yield / after-yield split possible — anything before `yield` is startup, anything after is shutdown.

***Likely questions:** What does `lifespan.py` do? · What does `@asynccontextmanager` give you here?*

yield in dependencies — the resource pattern

In a FastAPI dependency, `yield` is how you hand a resource (a database session, a transaction, a file) to the route and then clean it up after the request finishes — even if it raised an exception. The classic shape is: open a session, `yield` it, close it in a `finally` block. This is why you almost never see `session.close()` scattered through your route code — the dependency does it for you.

Likely questions: *Why does your `get_db` dependency use `yield` instead of `return`? · What happens if the route raises an exception?*

Did you actually use Depends?

Dependency injection means FastAPI builds and hands you the things your route needs — a database session, the current user, an HTTP client. You declare what you want as a parameter with `= Depends(...)`, and the framework wires it up. If your routes manually construct a session or call `SessionLocal()` inside the function body, you are not using DI — you are working around it.

Show one specific endpoint and walk through every `Depends` in its signature. That is the question.

Likely questions: *Did you use dependency injection in your application? · Do you know what `Depends` is? (Hint: dependencies are provided to you.) · Show me a place where you used dependency injection.*

4. Agents, Tools, and Where the Pieces Live

An agent is an LLM in a loop with access to tools. The reviewer's questions test whether you actually understand the wiring — where tools are defined, how they are registered, who fills in their arguments, and what the executor does. "LangGraph handles it" is not an answer.

Defining your tools

Each tool is a function with a clear name, a docstring, and a typed argument schema. The schema (often a Pydantic model passed as `args_schema`) is what the LLM reads to decide whether the tool fits the user's request and what arguments to pass. A vague docstring or missing types means the model will guess — and guess wrong.

`coroutine` is the `async` function the framework will *execute* when the LLM picks the tool. `args_schema` is the contract for *what arguments* the function expects. Two different jobs.

Likely questions: *Where did you define the tools for the agent? · What is `args_schema`? What is `coroutine`? · Where are the tool arguments actually filled in?*

Registering tools with the agent

Defining a tool is not the same as giving it to the agent. There is a specific call where you pass your list of tools to the agent or graph — `create_react_agent(..., tools=[...])`, `ToolNode(tools)`, or similar. If you cannot point to that line, the agent does not know your tools exist.

Likely questions: *Where are you telling the agent that these are your tools? · How are you giving your tools to the agent?*

Tool node vs LLM node (LangGraph)

In LangGraph, the LLM node decides *what to do next* — call a tool or finish. The tool node actually *runs* the chosen tool with the arguments the LLM produced and returns the result back into the state. Two nodes, two responsibilities. If your graph only has an LLM node, your tools are decorative.

Likely questions: *What is the difference between the LLM node and the tool node? · What executor are you using?*

Show your prompts

The prompt is part of the system. It belongs in source control, not pasted into a chat window the night before the demo. Reviewers will ask to see your system prompt and the way you templated user input into it. Prompt engineering is engineering — version it, review it, and be ready to defend the wording.

Likely questions: *Show me your prompts. · Where are they stored and how are they templated?*

5. Async HTTP Clients — Why httpx, Why Async

httpx.AsyncClient and parallel calls

`httpx.AsyncClient` is the async HTTP client we use to talk to external APIs. The reason to reach for the async version is concrete: when an agent needs to call several APIs to answer one question — weather, flights, and a travel-plan service — we can fire all three requests at once with `asyncio.gather` and wait for the slowest one, instead of waiting for each in turn. Sequential takes the sum of the latencies. Parallel takes the max.

Open the client once at app startup (lifespan), share it across requests, and close it on shutdown. Creating a new client for each call defeats the connection pooling the library is designed for.

Likely questions: *What does `httpx.AsyncClient` do? · Why async instead of requests? · Where is the client created and where is it closed?*

6. Tracing the Agent — LangSmith

What tracing.py is doing

The `tracing.py` file (or whatever you named it) wires every tool and chain into LangSmith so each agent run becomes a tree of timed, inspectable steps in the LangSmith UI. The `@traceable` decorator is the per-function hook — it tells LangSmith to record this call as a span.

When something goes wrong in an agent — wrong tool picked, hallucinated arguments, infinite loop — the LangSmith trace is where you find it. `print()` in a tool function will not save you.

Likely questions: *Were you able to set up LangSmith tracing? · What does `@traceable` do? · What happens in your `tracing.py` file?*

7. API Contracts — Don't Leak Your Database

DTOs and response_model are the same idea

The database has more fields than the client should see — password hashes, internal flags, soft-delete timestamps. The **Data Transfer Object** pattern says: define a separate output shape for each endpoint and only include the fields the caller is allowed to see. In FastAPI, this is `response_model`. The name DTO is just what other ecosystems call the same idea.

If your `/users/me` endpoint returns the password hash, that is not a DTO problem — that is a leak.

***Likely questions:** Do you know the data transfer object pattern? · Why is response_model on this endpoint different from your DB model?*

Streaming responses (SSE)

A streaming response sends tokens to the client as they are generated, instead of buffering the entire reply and shipping it once at the end. The mechanism is server-sent events: an open HTTP connection where the server pushes chunks. This is how chat UIs feel instantaneous even though the model takes seconds to finish.

***Likely questions:** What is a streaming response? · How does it differ from a normal response?*

Generic[T] — typed containers

`Generic[T]` lets you write a class or function that works with many types while still telling the type-checker which one it is in this case. The common use is a generic envelope — `PaginatedResponse[User]`, `PaginatedResponse[AgentRun]` — one class, many concrete shapes. If you used it, be ready to explain why you needed it instead of two separate classes.

***Likely questions:** What is the Generic[T] parameter you used here? Why?*

8. ML Revisited — Leakage in This Week's Project

Where leakage shows up in agentic projects

We covered leakage in earlier weeks for ML pipelines. This week the same problem reappears in a new disguise: features computed across the full dataset before splitting, evaluation data ending up in a retrieval index that the agent later queries, prompts that include ground-truth answers "just for testing." Leakage is not an ML-only problem — any pipeline where information from the answer slips into the question is leaking.

If you flagged leakage in your README or notebook, be ready to point at the exact place in the code where it happens, and the exact fix.

***Likely questions:** How is the leakage you specified in your application happening here? · What did you change to fix it?*

9. Concepts Worth Knowing By Name

Query rewriting

Query rewriting is the automatic transformation of a query — a database query, a search query, or a user's natural-language question — into an equivalent form that is faster or more retrievable. In SQL, the planner rewrites your query for performance. In RAG, you rewrite vague user questions into more specific search queries before hitting the vector store. Same idea, different layer.

***Likely questions:** What is query rewriting? Where could it help in your RAG pipeline?*

10. How You Talk About Your Code

Don't sabotage yourself in the review

We covered "defending your code" in Week 3. Two specific verbal habits keep showing up and they undermine you every time:

- **"I think..."**, **"I believe..."**, **"maybe..."** in front of a fact you actually know. The reviewer hears "I am not sure" and starts doubting the rest of your answer. If you know it, state it.
- Reading code to the reviewer instead of explaining it. They can read. Tell them *what* the code does and *why* it is there.

Good engineers say "I don't know — let me check" when they don't, and state facts plainly when they do. Both are stronger than hedging.

Pre-Review Checklist for Week 4

- I can walk through registration end-to-end and point at the line that writes to the database.
- I have opened pgAdmin (or equivalent) and seen my data in the actual table.
- Restarting my containers does not lose my data.
- Every protected endpoint has a Depends() that requires a valid token.
- I can show the Authorization: Bearer header in my client and the dependency that parses it.
- I know what my app returns when the token is missing (401) vs when the user lacks permission (403).
- I can describe what happens when an access token expires, and what a refresh token does.
- I have an Alembic migration committed for every schema change.
- I can explain upgrade vs downgrade and why we don't just delete the volume.
- Every tool has a typed args_schema and a clear docstring.
- I can point at the exact line where my tools are passed to the agent.
- I can explain the difference between the LLM node and the tool node in my graph.
- My system prompt is in source control, not in a chat window.
- LangSmith shows traces for my agent runs, and I have looked at one this week.

- My response_model on each endpoint is different from my ORM model where it needs to be.
- If I used Generic[T], httpx.AsyncClient, or streaming responses, I can say in one sentence why.